

Lab 10: Genetic Algorithm – Function Maximization

Aim

To implement a **Genetic Algorithm (GA)** to find the maximum value of a mathematical function.

Algorithm

1. Initialize a population of random solutions.
2. Calculate the fitness value of each solution.
3. Select the best solutions as parents.
4. Generate new solutions using crossover.
5. Apply mutation to introduce variation.
6. Repeat the process for a fixed number of generations.
7. Display the best solution and its maximum fitness value.

Program

Lab 10: Genetic Algorithm – Function Maximization

```
import random
```

```
# Function to maximize
```

```
def fitness(x):
```

```
    return x * x
```

```
# Create initial population
```

```
population = [random.randint(0, 31) for _ in range(10)]
```

```
# Run Genetic Algorithm
```

```
for generation in range(20):
```

```
# Sort population based on fitness
```

```
population = sorted(
```

```
    population,
```

```
    key=fitness,
```

```
    reverse=True
```

```
)
```

```
# Select the best two parents
```

```
parent1 = population[0]
```

```
parent2 = population[1]
```

```
# Create new population
```

```
new_population = [parent1, parent2]
```

```
while len(new_population) < 10:
```

```
    # Crossover
```

```
    child = (parent1 + parent2) // 2
```

```
    # Mutation
```

```
    if random.random() < 0.2:
```

```
        child += random.randint(-3, 3)
```

```
    # Keep value within range
```

```
    child = max(0, min(31, child))
```

```
    new_population.append(child)
```

```
population = new_population
```

```
# Find the best solution
```

```
best = max(population, key=fitness)
```

```
print("Best Solution:", best)
```

```
print("Maximum Fitness:", fitness(best))
```

Sample Output

Best Solution: 31

Maximum Fitness: 961

The exact output may vary slightly because the initial population and mutation are random.

Result

Thus, the **Genetic Algorithm was successfully implemented** to maximize the function $f(x) = x^2$, and the best solution was obtained.